UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ingeniería Industrial y de Sistemas · Escuela de Ingeniería de Software

GUÍA PRÁCTICA — SEMANA 5

Arquitectura de software

SW705 — Taller de Ingeniería de Software I

Docente: Mg. Ing. José Carlos, García La Riva

Caso aplicado: Clínica San Rafael S.A.C.

Integrantes del equipo	Jhostin Leonardo Rodriguez Neyra Rafael Olivos Gallardo Carlos Benjamin Fernandez Iquise
Código(s)	20231172F
Sección	U
Fecha	01/10/26

Instrucciones generales

Esta guía acompaña la Semana 5 del curso y los conduce, paso a paso, a diseñar la arquitectura inicial del sistema de información de su proyecto integrador. Cada paso incluye un recuadro de orientación (🧭), un ejemplo resuelto sobre el caso Clínica San Rafael S.A.C. (✅) y un espacio en blanco para que ustedes lo completen con su propio proyecto.

Este producto es un avance formativo hacia el Avance 3 del curso (arquitectura de software, modelo de base de datos y prototipo UX/UI), que se cierra en la Semana 6. Al final de esta guía encontrarán la rúbrica de revisión formativa de esta sesión.

> 🧭 **Antes de empezar**
>
> Tengan a la mano el backlog priorizado y los diagramas UML (casos de uso, clases, actividades) elaborados en la Semana 4: la arquitectura debe ser coherente con ellos.
>
> Trabajen en equipo: la arquitectura es una decisión colectiva, no de una sola persona.
>
> El caso Clínica San Rafael S.A.C. se usa solo como ejemplo guía — desarrollen la arquitectura de SU propio proyecto.

Paso 1 — Selección del estilo arquitectónico

Objetivo

Elegir y justificar el estilo o patrón arquitectónico más adecuado para su proyecto, comparando al menos dos alternativas.

> 🧭 **Cómo hacerlo**
>
> Listen los estilos candidatos (capas, cliente-servidor, MVC, microservicios, etc.).
>
> Para cada uno, evalúen: complejidad de implementación, tamaño del equipo, tiempo disponible (un semestre) y necesidad real de escalar por módulos.
>
> Elijan uno y expliquen por qué es mejor que la alternativa descartada para SU proyecto — no existe una respuesta única válida para todos los equipos.

> ✅ **Ejemplo — Clínica San Rafael S.A.C.**
>
> Alternativa A: Arquitectura en capas con API REST monolítica.
>
> Alternativa B: Microservicios (uno por módulo: HCE, Agenda, Facturación...).
>
> Decisión: se elige la Alternativa A, porque el equipo es pequeño (4 integrantes), el proyecto dura un semestre y aún no existe necesidad de escalar u operar los módulos de forma independiente. Los microservicios se dejan como evolución futura, no como requisito del prototipo académico.

Complete con su proyecto

Alternativa evaluada	Ventajas	Desventajas / riesgos	¿Se elige? (Sí/No)
Arquitectura en capas con API REST monolítica (presentación, lógica de negocio, acceso a datos, BD; frontend web consumiendo la API)	Una sola unidad de despliegue y de base de datos, lo que facilita la integridad transaccional entre HCE, citas y facturación (por ejemplo, impedir DNI duplicado o validar disponibilidad consolidada). Separación clara de responsabilidades, que facilita aplicar permisos por rol y bitácora de auditoría en un solo punto (RNF02). Es simple de probar para RNF01 (una sola API, un endpoint de HCE). La misma API sirve a admisión, médicos y, después, al portal del paciente (M6). Curva de aprendizaje baja para un equipo estudiantil. Permite modularizar internamente.	Si crece mucho, el monolito se acopla y escala como un bloque. Una falla en la API afecta todos los módulos (mitigable con backups diarios, RNF07, y monitoreo de salud, RNF03). Exige disciplina para no mezclar capas ni romper los límites entre módulos.	Sí
Microservicios (HCE, Agenda, Laboratorio, Facturación, Reportes, Portal como servicios independientes)	Escalado y despliegue independientes por módulo. Aislamiento de fallos (por ejemplo, que caiga Reportes sin afectar Agenda). Equipos pueden trabajar en paralelo en tecnologías distintas.	Complejidad desproporcionada, varias bases de datos, comunicación entre servicios, consistencia eventual, gateway, observabilidad y despliegue. Las reglas que cruzan módulos (cita ligada a paciente, orden de laboratorio ligada a consulta, cobro ligado a convenio) se vuelven transacciones distribuidas. Con 8 000 pacientes y una sola clínica no hay carga que lo justifique. Riesgo alto de no terminar en un semestre. Más superficie de ataque y más puntos que auditar para la Ley N.° 29733.	No
MVC monolítico con vistas renderizadas en servidor	Muy rápido de construir, un solo proyecto.	Acopla interfaz y lógica, dificultando exponer la misma lógica al portal del paciente (M6) y a futuros canales como WhatsApp o app móvil. Menos flexible para una interfaz simple y reactiva (RNF05).	No

Alternativa evaluada	Ventajas	Desventajas / riesgos	¿Se elige? (Sí/No)
Cliente-servidor de 2 capas (cliente pesado con acceso directo a la BD)	Muy simple.	La lógica de negocio queda en el cliente o en la BD, lo que debilita el control de acceso por rol y la bitácora (RNF02), complica el cifrado y no sirve para el acceso web del paciente.	No

Decisión final y justificación:

Se elige la arquitectura en capas con API REST monolítica modular porque:

1. Escala del problema. La clínica es una sola sede con hasta 8 000 pacientes (RNF01). No existe hoy una necesidad real de escalar o desplegar los módulos de forma independiente, que es lo único que justifica el costo de los microservicios.
2. Integridad de las reglas clínicas. Varios requisitos Must dependen de operaciones consistentes dentro de una sola transacción: no duplicar DNI (RF01), anular en vez de eliminar (RF04, RNF04), validar disponibilidad consolidada al agendar y liberar el horario al reprogramar o cancelar (RF05, RF07). En un monolito con una sola base de datos esto se resuelve con transacciones ACID normales; en microservicios exigiría patrones complejos (sagas, consistencia eventual) en un dominio donde un error afecta la seguridad del paciente.
3. Seguridad y cumplimiento centralizados. Con una sola API, el control de acceso por rol y la bitácora de auditoría (RNF02), junto con el cifrado en tránsito y en reposo (RNF06), se implementan una vez en una capa transversal, en lugar de replicarlos y auditarlos en seis servicios. Esto reduce el riesgo frente a la Ley N.° 29733.

Paso 2 — Atributos de calidad

Objetivo

Definir, con métricas concretas, los atributos de calidad que la arquitectura debe garantizar.

> **🧭 Cómo hacerlo**
>
> No basta con decir "debe ser rápido" o "debe ser seguro": cada atributo necesita una métrica medible.
>
> Prioricen 3 a 5 atributos que sean realmente críticos para su proyecto — no todos los atributos posibles aplican con la misma fuerza.
>
> Relacionen cada atributo con un riesgo concreto de no cumplirlo.

> **✅ Ejemplo — Clínica San Rafael S.A.C.**
>
> Rendimiento: la agenda de citas debe responder en menos de 2 segundos con 500 pacientes activos.
>
> Seguridad: acceso por roles (recepción, médico, administración) y cifrado de contraseñas.
>
> Disponibilidad: el sistema debe operar durante el horario de atención (7:00 a. m. – 9:00 p. m.), con respaldo diario de la base de datos.
>
> Modificabilidad: debe ser posible agregar el módulo de Facturación sin modificar el código de HCE ni de Agenda.

Complete con su proyecto

Atributo de calidad	Métrica / criterio medible	Riesgo si no se cumple
Seguridad (RNF02, RNF06, RNF04)	Acceso por rol: matriz de permisos con 4 roles (admisión, médico, enfermería, gerencia); el 100 % de los intentos de acceso no autorizado en la prueba es rechazado. Auditoría: el 100 % de las lecturas y ediciones de la HCE queda registrado en bitácora con usuario, fecha y acción. Cifrado: todo el tráfico viaja por HTTPS/TLS y los datos clínicos se almacenan cifrados (at-rest), verificado sobre un registro de prueba. Cifrado de contraseñas con hash salteado (no reversible).	Filtración o acceso indebido a datos de salud (datos sensibles bajo la Ley N.° 29733), con sanciones para la clínica, pérdida de confianza de los pacientes y responsabilidad legal.
Rendimiento (RNF01)	La HCE abre en menos de 2 segundos con una base de 8 000 pacientes, medido en al menos 20 repeticiones del endpoint de consulta de historia clínica bajo prueba de carga (criterio sugerido: promedio y percentil 95 por debajo de 2 s).	El médico espera con el paciente en consulta, se alarga el tiempo de espera y el personal vuelve al papel/Excel, con lo cual el sistema fracasa en su adopción.
Disponibilidad (RNF03, RNF07)	Agenda y emergencia con indisponibilidad no planificada menor al 1 % durante el horario de atención, medida con health-check periódico y registro de incidentes. Backup diario automático de la base de datos, con prueba de restauración exitosa en ambiente de prueba (integridad verificada) y pérdida máxima de datos de 24 h.	Recepción no puede agendar ni atender emergencias, se pierden citas y se paraliza la clínica. Ante una falla de disco o corrupción de datos se pierden historias clínicas, con consecuencias clínicas y legales graves.

Atributo de calidad	Métrica / criterio medible	Riesgo si no se cumple
Usabilidad (RNF05)	En una prueba con 3 usuarios (nivel de alfabetización digital bajo, medio y alto), el 100 % completa el agendamiento sin soporte técnico, con un máximo de 2 errores recuperables por tarea.	El personal con menor nivel digital rechaza el sistema, se siguen usando papel y llamadas, y el 18 % de inasistencia no baja.
Modificabilidad (RNF05)	Agregar o extender un módulo (por ejemplo, M4 Facturación o M6 Portal) debe lograrse sin modificar el código interno de M1 ni M2; los módulos se comunican solo por interfaces de servicio, nunca accediendo a las tablas de otro módulo.	Cada módulo nuevo rompe lo ya construido, se acumula deuda técnica.

Paso 3 — Decisiones de arquitectura (ADR)

Objetivo

Redactar al menos 3 registros de decisión de arquitectura (Architecture Decision Records) para las decisiones más relevantes del proyecto.

> **🧭 Estructura de un ADR**
>
> Contexto: ¿qué problema u opción se está evaluando?
>
> Alternativas consideradas.
>
> Decisión tomada.
>
> Consecuencias: ¿qué se gana y qué se sacrifica con esta decisión?

> **✅ Ejemplo — Clínica San Rafael S.A.C. (ADR-01)**
>
> Contexto: se necesita elegir el estilo arquitectónico general del sistema.
>
> Alternativas: arquitectura en capas vs. microservicios.
>
> Decisión: arquitectura en capas con API REST.
>
> Consecuencias: despliegue más simple y menor curva de aprendizaje para el equipo; en el futuro, separar módulos en microservicios implicará un esfuerzo de refactorización.

Complete con su proyecto — ADR 1

Contexto	Alternativas consideradas	Decisión	Consecuencias
Se debe elegir el estilo general del sistema para una clínica de una sola sede (hasta 8 000 pacientes), con un equipo pequeño y un semestre de plazo. Varias reglas Must cruzan módulos (cita ligada a paciente, orden de laboratorio ligada a consulta, cobro ligado a convenio).	- Capas + API REST monolítica modular. - Microservicios por módulo. - MVC con vistas en servidor. - Cliente-servidor de 2 capas.	Arquitectura en capas (presentación, lógica de negocio, acceso a datos) con API REST monolítica, organizada internamente en módulos M1-M6 que se comunican solo por interfaces de servicio.	Se gana: una sola unidad de despliegue, transacciones ACID entre módulos, seguridad y auditoría implementadas una sola vez, y pruebas de los RNF sobre un único sistema. Se sacrifica: escalado independiente por módulo y aislamiento de fallos. Si en el futuro se separan módulos (por ejemplo, el portal M6) como microservicios, habrá un esfuerzo de refactorización, que se reduce si se respeta el desacoplamiento interno desde el inicio.

Complete con su proyecto — ADR 2

Contexto	Alternativas consideradas	Decisión	Consecuencias
La HCE, las citas y la facturación manejan datos fuertemente relacionados y requieren integridad estricta: no duplicar DNI (RF01), validar disponibilidad consolidada (RF05), anular sin borrar (RF04). Además se exige abrir la HCE en menos de 2 s con 8 000 pacientes (RNF01) y cifrado de datos clínicos en reposo (RNF06).	- Base relacional (PostgreSQL o MySQL). - Base documental (MongoDB), con la HCE como documento por paciente. - Una base por módulo (persistencia políglota).	Una única base de datos relacional (PostgreSQL), con esquemas lógicos separados por módulo, restricción UNIQUE sobre el número de documento, claves foráneas entre entidades e índices por DNI, por paciente y por fecha de episodio.	Se gana: Integridad referencial y transacciones que garantizan las reglas clínicas, restricciones declarativas (DNI único) que no dependen del código, buen rendimiento con 8 000 pacientes mediante índices, y soporte nativo de cifrado en reposo y backups programados (RNF07). Se sacrifica: flexibilidad para estructuras clínicas muy variables por especialidad (se mitiga con campos JSONB para datos específicos de cada especialidad) y la posibilidad de escalar bases por módulo, que hoy no se necesita. Los módulos no deben acceder a las tablas de otro esquema, solo a través de interfaces de servicio.

Complete con su proyecto — ADR 3

Contexto	Alternativas consideradas	Decisión	Consecuencias
La HCE es un documento médico-legal. Ninguna entrada puede eliminarse físicamente; solo se anula registrando motivo y usuario responsable (RF04, RNF04). Además, cada consulta o edición debe quedar registrada con usuario, fecha y acción (RNF02, Ley N.° 29733).	- Borrado lógico en las tablas (campo de estado “anulado”, motivo, usuario, fecha) más bitácora de auditoría en tabla aparte. - Borrado físico con copia del registro en una tabla histórica. - Event sourcing (solo se agregan eventos, el estado se reconstruye).	Borrado lógico, las entradas de HCE tienen estado activo/anulado, con motivo y usuario obligatorios al anular. La API no expone ninguna operación DELETE sobre la HCE, y el usuario de base de datos de la aplicación no tiene permiso DELETE sobre esas tablas. Una tabla de bitácora de solo inserción registra usuario, fecha/hora, acción y entidad afectada, tanto para lecturas como para ediciones, y se alimenta desde la capa transversal de la API.	Se gana: cumplimiento de RF04/RNF04 en dos niveles (aplicación y base de datos), trazabilidad médico-legal completa y una prueba de verificación simple (intentar borrar y comprobar que solo cambia el estado). Se sacrifica: las consultas deben filtrar por estado (se mitiga con vistas o filtros por defecto), la base crece más rápido (aceptable para 8 000 pacientes) y la bitácora de lecturas añade escrituras en cada acceso, lo que debe medirse en la prueba de rendimiento de RNF01. Event sourcing se descarta por su complejidad desproporcionada para el semestre.

Complete con su proyecto — ADR 4

Contexto	Alternativas consideradas	Decisión	Consecuencias
El acceso a la HCE debe restringirse por rol: admisión, médico, enfermería y gerencia (RNF02). Solo el médico tratante edita la HCE; enfermería tiene lectura y registro de signos vitales (RF09). Los datos de salud son sensibles bajo la Ley N.° 29733, y el tráfico debe ir cifrado (RNF06).	- RBAC con permisos validados en la capa de servicios de la API, sesión mediante token firmado (JWT con expiración corta) y contraseñas con hash salteado (bcrypt o Argon2). - Control de acceso solo en la interfaz (ocultar botones según el rol). - Proveedor de identidad externo (Keycloak, Auth0).	RBAC validado en el servidor, con una matriz de permisos rol-acción-recurso aplicada en un middleware o capa transversal antes de llegar a la lógica de negocio. Autenticación con token firmado de corta duración, contraseñas con hash salteado y comunicación solo por HTTPS/TLS. Cada decisión de acceso (permitido o denegado) alimenta la bitácora del ADR-03.	Se gana: un único punto de control y auditoría, una matriz de permisos fácil de revisar (verificación de RNF02) y protección real incluso si alguien invoca la API directamente. Se sacrifica: se debe diseñar y mantener la matriz de permisos y probar los casos de acceso denegado; la gestión de sesiones y expiración suma complejidad. Un proveedor externo se descarta para el prototipo por dependencia y curva de aprendizaje, y queda como evolución si se requiere inicio de sesión único o autenticación de dos factores.

Complete con su proyecto — ADR 5

Contexto	Alternativas consideradas	Decisión	Consecuencias
La clínica tiene 18 % de inasistencia, y RF06 exige confirmación y recordatorio automático por SMS, correo o WhatsApp. El envío depende de servicios externos que pueden fallar o demorar, y no debe afectar la disponibilidad de la agenda (RNF03).	- Envío sincrónico dentro de la petición de agendamiento. - Cola de mensajes interna más un proceso programado, que se conecta a proveedores externos (correo SMTP, SMS, WhatsApp Business API) mediante un adaptador. - Servicio de mensajería propio (módem SMS, servidor de correo propio).	Envío asíncrono, al registrar, reprogramar o cancelar una cita se crea una tarea de notificación en una cola (tabla en la misma base de datos para el prototipo). Un proceso programado (job) la procesa y la envía mediante un adaptador de canal intercambiable, con reintentos y registro del estado del envío. Los recordatorios se generan con anticipación configurable (por ejemplo, 24 h antes). Se prioriza correo y SMS en el prototipo, y WhatsApp queda detrás del mismo adaptador.	Se gana: si el proveedor falla, la agenda sigue funcionando (protege RNF03), se pueden reintentar envíos, y cambiar de canal no afecta la lógica de citas. Se sacrifica: hay un componente más (el job) que monitorear, hay costos recurrentes por mensaje con proveedores externos, y los mensajes pueden llegar con ligero retraso. Los mensajes deben incluir el mínimo de datos clínicos posible (fecha, hora, médico y especialidad, sin diagnóstico) por privacidad. WhatsApp requiere aprobación de plantillas del proveedor, lo que puede demorar su activación.

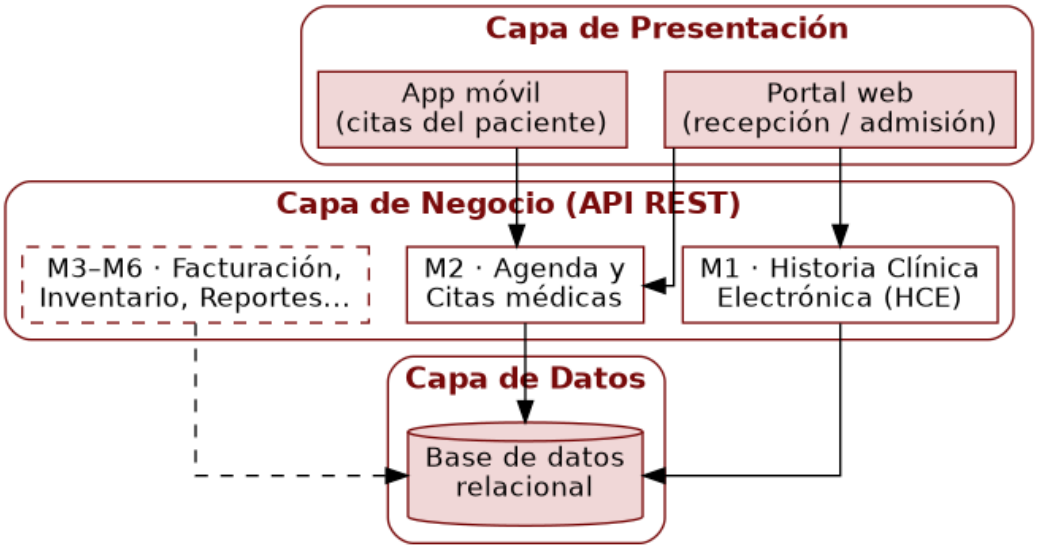
Paso 4 — Vistas del Documento de Arquitectura (SAD)

Objetivo

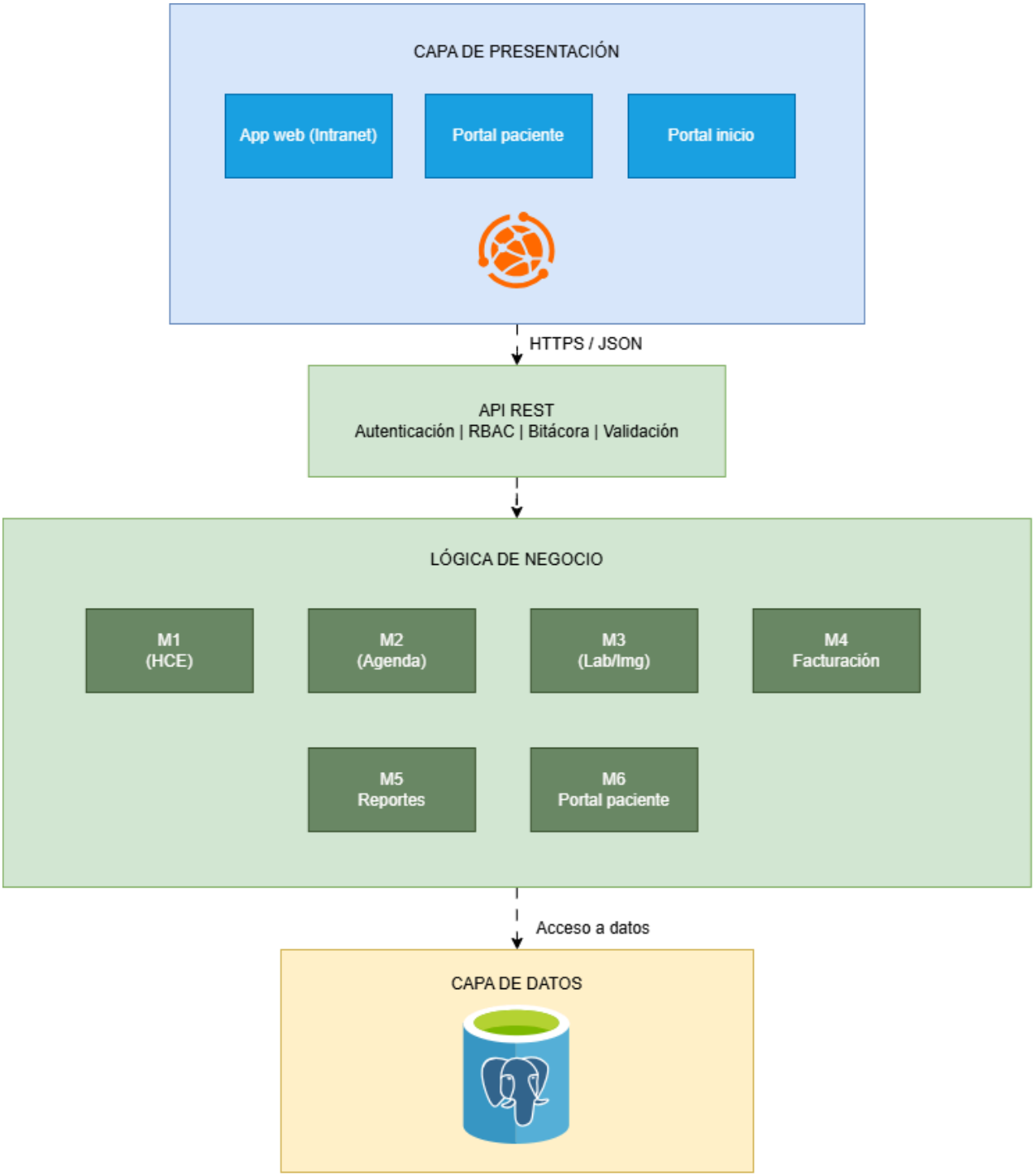
Elaborar, al menos, la vista lógica y la vista física (despliegue) de la arquitectura del sistema.

> 🧭 **Cómo hacerlo**
>
> Vista lógica: dibujen las capas o módulos del sistema y cómo se conectan entre sí.
>
> Vista física: dibujen en qué equipos/servidores correrá cada componente y cómo se comunican (protocolo).
>
> Usen una herramienta de diagramación (draw.io, Lucidchart, o incluso a mano de forma ordenada) y péguenla en el espacio indicado.

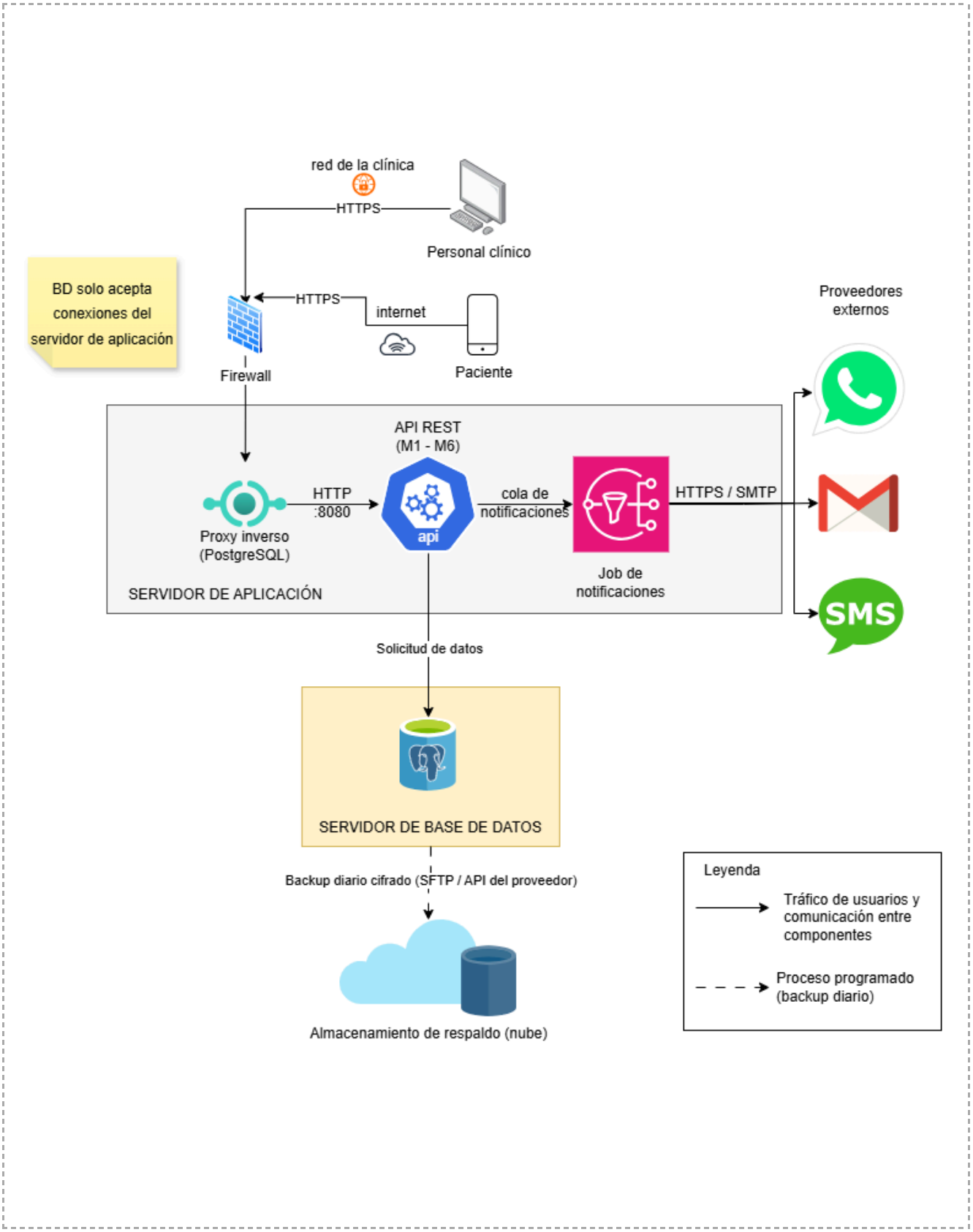
> ✅ **Ejemplo — Vista lógica en capas (Clínica San Rafael S.A.C.)**



Complete con su proyecto — Vista lógica


CAPA DE PRESENTACIÓN
App web (Intranet)
Portal paciente
Portal inicio
HTTPS / JSON
API REST
Autenticación | RBAC | Bitácora | Validación
LÓGICA DE NEGOCIO
M1
(HCE)
M2
(Agenda)
M3
(Lab/Img)
M4
Facturación
M5
Reportes
M6
Portal paciente
Acceso a datos
CAPA DE DATOS

Complete con su proyecto — Vista física (despliegue)



Paso 5 — Stack tecnológico

Objetivo

Seleccionar y justificar el stack tecnológico (lenguaje, framework, base de datos, hosting) que usará el equipo para construir el proyecto.

🧭 Cómo hacerlo

Consideren la experiencia previa del equipo: un stack conocido reduce el riesgo del proyecto.

Verifiquen que el stack elegido soporte los atributos de calidad definidos en el Paso 2.

Prefieran herramientas gratuitas o de nivel académico (planes free/estudiante) para el despliegue.

✅ Ejemplo — Clínica San Rafael S.A.C.

Lenguaje/backend: Python con FastAPI (rapidez de desarrollo, buena documentación automática).

Base de datos: SQLite en desarrollo, con posibilidad de migrar a PostgreSQL en producción sin cambiar el código del modelo.

Frontend: por definir en la Semana 6, junto con el prototipo UX/UI.

Control de versiones y CI: GitHub + GitHub Actions para pruebas automáticas.

Complete con su proyecto

Componente	Tecnología elegida	Justificación
Lenguaje	Node.js con TypeScript	Un solo lenguaje en backend y frontend, ecosistema enorme y curva de aprendizaje baja para el equipo
Framework backend	NestJS	NestJS impone una estructura en módulos (un módulo por M1-M6), con controladores, servicios y repositorios: es la arquitectura en capas casi lista.
Base de datos	PostgreSQL	Igual que antes: UNIQUE para DNI, ACID, JSONB, cifrado en reposo
Acceso a datos	Prisma	Modelo de datos declarativo, migraciones versionadas y consultas tipadas que reducen el riesgo de inyección SQL.
Validación	class-validator	Valida DNI, fechas, códigos CIE-10 antes de llegar a la lógica de negocio
Autenticación y seguridad	JWT de corta duración (jsonwebtoken) + bcrypt + Guards de NestJS para RBAC + helmet	Implementa la Decisión 04: matriz de permisos validada en el servidor antes de la lógica de negocio. Helmet agrega cabeceras de seguridad
Bitácora de auditoría	Interceptor de NestJS que registra usuario, fecha y acción en la tabla de solo inserción	Un único punto transversal para RNF02, sin repetir código en cada módulo

Componente	Tecnología elegida	Justificación
Frontend	React con Vite	Mismo lenguaje que el backend; SPA simple y reactiva (RNF05). Para el diseño: Tailwind
Documentación de la API	Swagger (@nestjs/swagger)	Documentación automática, útil para el equipo y para la demostración
Proxy inverso	Nginx	Certificados automáticos (RNF06)
Notificaciones	BullMQ, correo con Nodemailer; SMS/WhatsApp con proveedor	Coincide con la Decisión 05
Contenedores	Docker + Docker Compose	Mismo entorno en desarrollo y despliegue
Hosting	VM en nube con plan gratuito o estudiante (Oracle Cloud Free Tier, créditos del GitHub Student Pack) o Render/Railway en nivel gratuito	Bajo costo y coherente con la vista física
Respaldo	pg_dump programado (cron) + subida cifrada a almacenamiento en la nube	Cumple RNF07; se prueba la restauración
Monitoreo	Endpoint /health (@nestjs/terminus) + UptimeRobot	Mide RNF03 con registro de incidentes
Pruebas	Jest + Supertest (unitarias e integración de la API)	Cubren la verificación de RNF01 (20 repeticiones con 8 000 pacientes), RNF02 y RNF04
Control de versiones y CI	GitHub + GitHub Actions	Pruebas automáticas por commit, ramas por módulo, pull requests

Rúbrica de evaluación — Revisión formativa, Semana 5

Carácter: FORMATIVO. Esta revisión no otorga una nota que se promedie directamente en la Promedio de Prácticas (PP); su propósito es retroalimentar el avance del equipo antes del cierre del Avance 3 (arquitectura, modelo de base de datos y prototipo UX/UI) en la Semana 6, donde sí se evaluará con una rúbrica sumativa.

Criterio	Logrado (18–20)	Satisfactorio (14–17)	En proceso (11–13)	Inicial (0–10)
Selección y justificación del estilo arquitectónico (25%)	Compara alternativas reales y justifica la decisión con criterios técnicos sólidos y propios del proyecto.	Compara alternativas y justifica la decisión, con criterios generales.	Elige un estilo sin comparar alternativas o con justificación superficial.	No presenta estilo arquitectónico definido.
Atributos de calidad medibles (25%)	Define 3 a 5 atributos con métricas claras y su riesgo asociado, pertinentes al proyecto.	Define atributos con métricas, aunque alguna es poco medible.	Menciona atributos de forma genérica, sin métricas.	No define atributos de calidad.
Calidad de los ADR (25%)	Presenta 3 ADR completos y coherentes (contexto, alternativas, decisión, consecuencias) para decisiones realmente relevantes.	Presenta 3 ADR completos, aunque alguna decisión es de bajo impacto.	Presenta menos de 3 ADR o con estructura incompleta.	No presenta ADR.
Coherencia de las vistas del SAD (lógica y física) (25%)	Ambas vistas son coherentes entre sí, con el backlog/UML de la Semana 4 y con el estilo elegido.	Ambas vistas están presentes, con alguna inconsistencia menor.	Presenta solo una vista o con inconsistencias relevantes.	No presenta vistas de arquitectura.

Escala vigesimal (0–20). El docente registrará el nivel alcanzado por criterio y brindará retroalimentación oral o escrita para que el equipo ajuste su arquitectura antes de la sustentación del Avance 3.